

1. lưu ý.

Chưa triển khai trong giai đoạn này:

AI/NLP Detector

TF-IDF training

ML model training

Vector search

Deep learning

Realtime model inference

=> Tuy nhiên, pipeline phải thiết kế sao cho sau này có thể gắn thêm các module trên dễ dàng.

2. Input và Output tổng thể

Input

Hệ thống nhận input là file access log có sẵn:

Apache access.log

Nginx access.log

IIS W3C log

Ví dụ:

```
127.0.0.1 - - [26/May/2026:10:15:32 +0700] "GET /index.php?id=1 HTTP/1.1" 200 1234 "-" "Mozilla/5.0"
```

Output

Hệ thống cần xuất ra các loại output:

outputs/

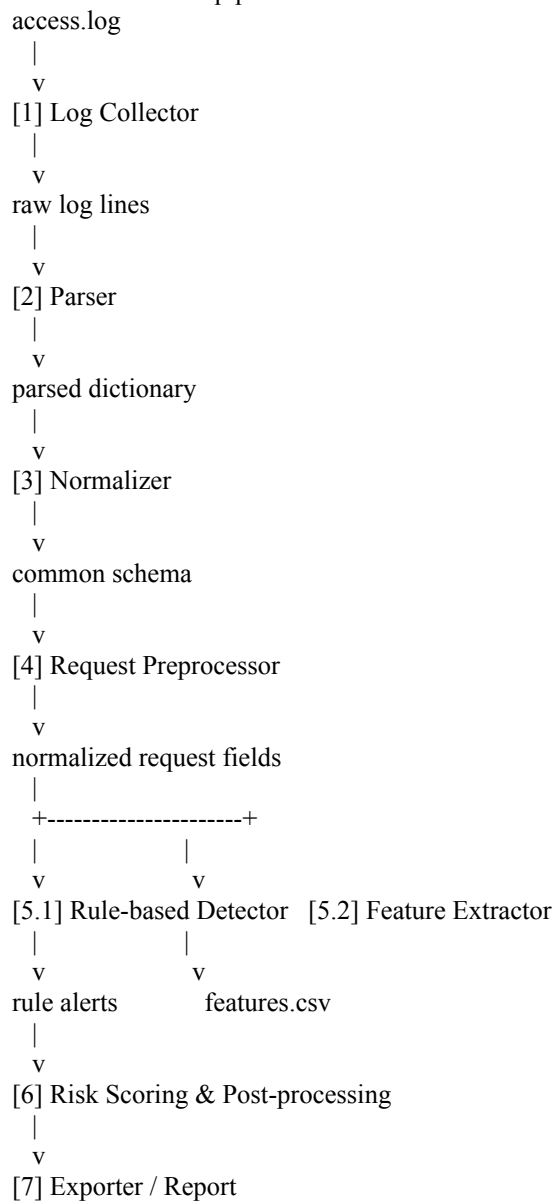
- raw_lines.jsonl
- parsed_logs.jsonl
- normalized_logs.jsonl
- normalized_logs.csv
- preprocessed_requests.jsonl
- features.csv
- alerts.jsonl
- alerts.csv
- report.md
- run_summary.json

ý nghĩa:

File	Mục đích
raw_lines.jsonl	Lưu raw log lines đã đọc từ collector
parsed_logs.jsonl	Lưu kết quả parse từng dòng
normalized_logs.jsonl	Lưu log theo common schema
normalized_logs.csv	Dữ liệu chuẩn hóa dạng CSV
preprocessed_requests.jsonl	Request sau khi decode, lowercase, tách path/query
features.csv	Feature thủ công phục vụ phân tích hoặc train ML sau này

alerts.jsonl	Alert do rule-based detector sinh ra
alerts.csv	Alert dạng bảng
report.md	Báo cáo tổng hợp
run_summary.json	Thống kê số dòng xử lý, parse success/fail, số alert

3. Kiến trúc pipeline:



Thiết kế phải theo kiểu module độc lập.

Mỗi module nên có input/output rõ ràng để dễ test riêng.

4. Module chi tiết

[1] Log Collector

Mục tiêu

Đọc file access log gốc và xuất ra danh sách raw log lines.

Input

data/raw /apache/access.log
data/raw /nginx/access.log
data/raw /iis/u_ex260526.log

Output

Mỗi dòng output nên có dạng:

```
{  
  "source_file": "data/raw/apache/access.log",  
  "server_type": "apache",  
  "line_number": 1,  
  "raw_line": "127.0.0.1 - - [26/May/2026:10:15:32 +0700] \"GET /index.php?id=1 HTTP/1.1\" 200 1234 \"-\"  
  \"Mozilla/5.0\"",  
  "ingest_ts": "2026-05-26T10:15:35+07:00"  
}
```

Yêu cầu kỹ thuật

- Đọc file ở chế độ bytes nếu cần để tránh lỗi encoding.
- Thử decode bằng UTF-8 trước.
- Nếu lỗi, fallback sang Latin-1 để không crash.
- Không làm mất raw line gốc.
- Không bỏ dòng lỗi, mà phải ghi trạng thái lỗi nếu không decode được.
- Hỗ trợ nhiều file input.
- Hỗ trợ chỉ định server_type: apache, nginx, iis, hoặc auto.

Không làm

- Không detect attack trong collector.
- Không normalize.
- Không sửa nội dung raw line.

[2] Parser

Mục tiêu: Parse raw log line thành dictionary chứa các trường thô.

Input: Output từ Log Collector.

Output

Ví dụ output:

```
{  
  "source_file": "data/raw/apache/access.log",  
  "server_type": "apache",  
  "line_number": 1,  
  "parse_status": "success",  
  "timestamp_raw": "26/May/2026:10:15:32 +0700",  
  "source_ip": "127.0.0.1",  
}
```

```
"method": "GET",
"request_target": "/index.php?id=1",
"http_version": "HTTP/1.1",
"status_code": 200,
"response_size": 1234,
"referrer": "-",
"user_agent": "Mozilla/5.0",
"raw_line": "..."
}
```

Nếu parse lỗi:

```
{
"source_file": "data/raw/apache/access.log",
"server_type": "apache",
"line_number": 12,
"parse_status": "failed",
"parse_error": "regex_not_matched",
"raw_line": "..."
}
```

Field cần parse

Bắt buộc:

```
timestamp_raw
source_ip
http_method
request_target
http_version
status_code
response_size
user_agent
referrer
raw_line
server_type
source_file
line_number
parse_status
```

Tùy chọn nếu có:

```
host
username
bytes_sent
request_time
upstream_status
cookie
x_forwarded_for
```

Hỗ trợ format

Cần hỗ trợ tối thiểu:

```
Apache Combined Log Format
Nginx Combined Log Format
IIS W3C Extended Log Format
```

Không làm

- Không URL decode trong parser.
- Không gán label attack.
- Không tính risk score.
- Không train model.

Parser chỉ chịu trách nhiệm trích xuất trường.

[3] Normalizer

Mục tiêu

Chuyển parsed dictionary về common schema thống nhất để các log Apache/Nginx/IIS có cùng cấu trúc.

Input

Output từ Parser.

Output common schema

```
{
  "event_id": "uuid",
  "event_time": "2026-05-26T10:15:32+07:00",
  "ingest_time": "2026-05-26T10:15:35+07:00",
  "source_file": "data/raw/apache/access.log",
  "server_type": "apache",
  "line_number": 1,

  "source_ip": "127.0.0.1",
  "http_method": "GET",
  "url_original": "/index.php?id=1",
  "url_path": "/index.php",
  "query_string": "id=1",
  "http_version": "HTTP/1.1",

  "status_code": 200,
  "response_size": 1234,
  "referrer": "-",
  "user_agent": "Mozilla/5.0",

  "parse_status": "success",
  "normalization_status": "success",
  "raw_line": "..."
}
```

Yêu cầu

- Convert timestamp về ISO-8601 nếu có thể.
- Tách request_target thành:
 - url_original
 - url_path
 - query_string
- Giữ lại raw field gốc.
- Không xóa dòng parse failed.
- Dòng parse failed vẫn phải được ghi ra output với status tương ứng.

Không làm

- Không detect attack.
 - Không train model.
 - Không thay đổi nghĩa của URL.
-

[4] Request Preprocessor

Mục tiêu

Chuẩn hóa request để phục vụ rule-based detection và feature extraction.

Input

Output từ Normalizer.

Output

```
{
  "event_id": "uuid",
  "url_original": "/product?id=1%27%20OR%201%3D1",
  "url_decoded_once": "/product?id=1' OR 1=1",
  "url_decoded_twice": "/product?id=1' OR 1=1",
  "url_lower": "/product?id=1' or 1=1",
  "url_path": "/product",
  "query_string": "id=1%27%20OR%201%3D1",
  "query_params": {
    "id": "1' OR 1=1"
  },
  "user_agent_lower": "mozilla/5.0",
  "preprocess_status": "success"
}
```

Các bước xử lý

URL decode một lần.

URL decode lần hai nếu cần để phát hiện double encoding.

Lowercase bản dùng cho detection.

Tách path và query string.

Parse query parameters.

Chuẩn hóa user-agent về lowercase.

Giữ cả bản gốc và bản đã xử lý.

Yêu cầu quan trọng

Không được thay thế hoàn toàn raw URL bằng URL đã decode.

Luôn giữ:

```
url_original
url_decoded_once
url_decoded_twice
url_lower
```

query_string_original
query_params

Lý do: trong security log, raw payload rất quan trọng cho điều tra.

[5] Rule-based Detector

Mục tiêu

Xây dựng baseline detector dùng rule/signature để phát hiện request đáng ngờ.

Đây là detector chính trong giai đoạn pipeline nền.

AI/NLP Detector chưa triển khai ở giai đoạn này.

Input

Output từ Request Preprocessor.

Output

```
{  
  "event_id": "uuid",  
  "is_suspicious": true,  
  "attack_type": "sqli",  
  "matched_rules": [  
    {  
      "rule_id": "SQLI_001",  
      "rule_name": "SQL injection tautology",  
      "severity": "high",  
      "matched_field": "url_decoded_once",  
      "matched_value": "' or 1=1"  
    }  
  ],  
  "rule_score": 80  
}
```

Nhóm rule cần có

SQL Injection

Pattern ví dụ:

```
' or 1=1  
" or "1"="1  
union select  
select from  
sleep(  
benchmark(  
or 1=1  
--  
#  
information_schema
```

XSS

Pattern ví dụ:

```
<script
</script>
onerror=
onload=
javascript:
alert(
document.cookie
<img
<svg
```

Path Traversal

Pattern ví dụ:

```
../
..\
%2e%2e
/etc/passwd
windows/win.ini
boot.ini
```

Command Injection

Pattern ví dụ:

```
; cat
; ls
| whoami
&& id
`id`
$(id)
wget
curl
/bin/sh
cmd.exe
powershell
```

Scanner / Bot

Dựa vào user-agent hoặc URL:

```
sqlmap
nikto
acunetix
nmap
masscan
dirbuster
gobuster
wpscan
```

Brute-force / Scanning behavior

Rule theo hành vi đơn giản:

- N request lỗi 404 từ cùng IP trong khoảng thời gian ngắn
- N request đến nhiều path khác nhau từ cùng IP
- N request login failed nếu có status phù hợp

Giai đoạn đầu có thể triển khai rule hành vi đơn giản theo batch, chưa cần realtime phức tạp.

Không làm

- Không dùng ML model.
- Không dùng TF-IDF.
- Không dùng embedding.
- Không dùng vector search.

Rule-based detector chỉ dùng pattern và logic đơn giản.

[6] Hand-crafted Feature Extractor

Mục tiêu

Trích xuất feature thủ công từ request để:

1. Phân tích dữ liệu.
2. Xuất features.csv.
3. Chuẩn bị input cho module ML ở giai đoạn sau.

Không train model trong module này.

Input

Output từ Normalizer và Request Preprocessor.

Output

Một dòng feature tương ứng một request.

Ví dụ features.csv:

```
event_id,source_ip,http_method,status_code,url_length,path_length,query_length,num_params,num_digits,num_special_chars,num_percent_encoding,num_dotdot,num_slash,num_quote,num_angle_bracket,entropy,user_agent_length,has_suspicious_keyword,rule_score,is_suspicious,attack_type
```

Feature cần tính

Nhóm độ dài:

```
url_length
path_length
query_length
user_agent_length
```

Nhóm ký tự:

```
num_digits
num_letters
num_special_chars
num_percent_encoding
num_slash
num_dot
num_dotdot
num_quote
num_double_quote
num_angle_bracket
```

num_parentheses
num_semicolon
num_pipe
num_ampersand
num_equal

Nhóm cấu trúc:

num_params
path_depth
has_query
has_file_extension

Nhóm entropy:

url_entropy
query_entropy

Nhóm rule output:

has_suspicious_keyword
rule_score
is_suspicious
attack_type

Yêu cầu

- Feature extractor không phụ thuộc vào ML.
- Có thể chạy độc lập.
- Nếu trường thiếu, điền giá trị mặc định.
- Không crash khi URL rỗng hoặc parse failed.

[7] Risk Scoring & Post-processing

Mục tiêu

Tổng hợp kết quả từ rule-based detector và feature extractor để tạo risk score.

Input

normalized logs
preprocessed requests
rule detection result
hand-crafted features

Output

```
{  
  "event_id": "uuid",  
  "source_ip": "192.168.1.10",  
  "url_original": "/product?id=1' OR 1=1",  
  "is_suspicious": true,  
  "attack_type": "sql",  
  "severity": "high",  
  "risk_score": 85,  
  "reason": [  
    "Matched SQL injection tautology rule",  
    "High number of special characters",
```

```
"Contains quote and equality pattern"  
]  
}
```

Risk score đề xuất

0 - 29: low
30 - 69: medium
70 - 100: high

Ví dụ tính điểm:

matched high severity rule: +70
matched medium severity rule: +40
matched low severity rule: +20
scanner user-agent: +50
many 404 from same IP: +30
high special character count: +10
high entropy: +10

Giới hạn:

risk_score tối đa = 100

Dedup alert

Nếu nhiều alert giống nhau, có thể gộp theo:

source_ip
attack_type
url_path
time window

[8] Exporter / Report

Mục tiêu

Xuất dữ liệu để dùng cho phân tích, SIEM, hoặc training ML sau này.

Output bắt buộc

normalized_logs.jsonl
normalized_logs.csv
features.csv
alerts.jsonl
alerts.csv
report.md
run_summary.json

Nội dung report.md

Report cần có:

Tổng số dòng log
Số dòng parse thành công
Số dòng parse lỗi
Số request suspicious
Số alert theo attack_type

Top source_ip đáng ngờ
Top URL bị tấn công
Top user-agent đáng ngờ
Danh sách alert high severity

Format alert JSONL

```
{
  "event_time": "2026-05-26T10:15:32+07:00",
  "source_ip": "192.168.1.10",
  "http_method": "GET",
  "url_original": "/product?id=1' OR 1=1",
  "status_code": 200,
  "user_agent": "sqlmap/1.7",
  "attack_type": "sql",
  "severity": "high",
  "risk_score": 90,
  "matched_rules": ["SQLI_001", "SCANNER_001"]
}
```

5. CLI yêu cầu

Codex cần xây dựng CLI để chạy pipeline.

Ví dụ:

```
python -m src.main \
--input data/raw/apache/access.log \
--server-type apache \
--output-dir outputs/apache_run
```

Hỗ trợ nhiều file:

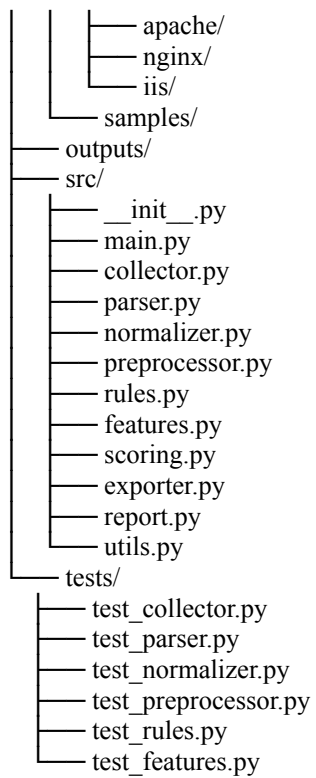
```
python -m src.main \
--input data/raw/apache/access.log data/raw/nginx/access.log \
--server-type auto \
--output-dir outputs/multi_run
```

Các tham số cần có:

```
--input
--server-type
--output-dir
--encoding
--rules-path
--time-zone
--export-jsonl
--export-csv
--generate-report
```

6. Cấu trúc thư mục đề xuất

```
project/
├── AGENTS.md
├── README.md
├── requirements.txt
├── configs/
│   ├── rules.yaml
│   └── pipeline.yaml
├── data/
└── raw/
```



7. Vai trò từng file code

Chịu trách nhiệm:

```
read_log_file()
read_multiple_files()
detect_or_apply_encoding()
emit_raw_line_records()
```

Chịu trách nhiệm:

```
parse_apache_line()
parse_nginx_line()
parse_iis_line()
parse_line_auto()
```

Chịu trách nhiệm:

```
normalize_parsed_record()
normalize_timestamp()
split_url_path_query()
build_common_schema()
```

Chịu trách nhiệm:

```
url_decode_once()
url_decode_twice()
lowercase_for_detection()
parse_query_params()
normalize_user_agent()
```

Chịu trách nhiệm:

```
load_rules()
match_sqli_rules()
match_xss_rules()
match_path_traversal_rules()
match_cmdi_rules()
match_scanner_rules()
detect_behavior_rules()
```

Chịu trách nhiệm:

```
extract_length_features()
extract_character_features()
extract_structure_features()
calculate_entropy()
build_feature_row()
```

Chịu trách nhiệm:

```
calculate_risk_score()
assign_severity()
deduplicate_alerts()
aggregate_by_ip()
```

Chịu trách nhiệm:

```
write_jsonl()
write_csv()
write_alerts()
write_summary()
```

Chịu trách nhiệm:

```
generate_markdown_report()
summarize_attack_types()
summarize_top_ips()
summarize_top_urls()
```

Chịu trách nhiệm điều phối pipeline:

```
collector
-> parser
-> normalizer
-> preprocessor
-> rule detector
-> feature extractor
-> scoring
-> exporter
-> report
    8. Rules config đề xuất
```

Tạo file:

configs/rules.yaml

Ví dụ:

```
rules:
- id: SQLI_001
  name: SQL injection tautology
  attack_type: sql_i
  severity: high
  fields:
    - url_lower
    - url_decoded_once
    - url_decoded_twice
  patterns:
    - "" or 1=1"
    - "\" or \"1\"=\"1"
    - " or 1=1"
    - ""=""

- id: SQLI_002
  name: SQL injection union select
  attack_type: sql_i
  severity: high
  fields:
    - url_lower
    - url_decoded_once
    - url_decoded_twice
  patterns:
    - "union select"
    - "union all select"
    - "information_schema"

- id: XSS_001
  name: XSS script tag
  attack_type: xss
  severity: high
  fields:
    - url_lower
    - url_decoded_once
    - url_decoded_twice
  patterns:
    - "<script"
    - "</script>"
    - "javascript:"
    - "onerror="
```

- "onload="
- id: PATH_001
 - name: Path traversal
 - attack_type: path_traversal
 - severity: high
 - fields:
 - url_lower
 - url_decoded_once
 - url_decoded_twice
 - patterns:
 - "../"
 - "..\\"
 - "%2e%2e"
 - "/etc/passwd"
 - "boot.ini"
- id: CMDI_001
 - name: Command injection
 - attack_type: command_injection
 - severity: high
 - fields:
 - url_lower
 - url_decoded_once
 - url_decoded_twice
 - patterns:
 - "; cat"
 - "; ls"
 - "| whoami"
 - "&& id"
 - "\$(id)"
 - "/bin/sh"
 - "cmd.exe"
 - "powershell"
- id: SCANNER_001
 - name: Scanner user agent
 - attack_type: scanning
 - severity: medium
 - fields:
 - user_agent_lower
 - patterns:
 - "sqlmap"
 - "nikto"
 - "acunetix"
 - "nmap"
 - "masscan"
 - "dirbuster"
 - "gobuster"
 - "wpscan"

9. Testing yêu cầu

Codex cần tạo unit test cho từng module.

Test Collector

Kiểm tra:

đọc được file UTF-8
fallback Latin-1
giữ line_number
giữ source_file
không làm mất raw_line

Test Parser

Kiểm tra:

parse Apache combined log
parse Nginx combined log
parse IIS W3C log
parse failed line không crash

Test Normalizer

Kiểm tra:

timestamp được convert đúng
request_target được tách path/query
field thiếu không làm crash
common schema đủ field

Test Preprocessor

Kiểm tra:

URL decode một lần
URL decode hai lần
lowercase
parse query params
giữ url_original

Test Rule-based Detector

Kiểm tra:

SQLi được detect
XSS được detect
Path traversal được detect
Command injection được detect
Scanner user-agent được detect
Benign request không bị flag quá nhiều

Test Feature Extractor

Kiểm tra:

url_length
query_length
num_special_chars
num_params
entropy
num_percent_encoding

Tiêu chí nghiệm thu

Hệ thống được coi là hoàn chỉnh ở giai đoạn pipeline nên nếu đạt các tiêu chí sau:

Chạy được CLI với file Apache/Nginx/IIS access log.

Parse được các trường quan trọng:

- a. timestamp
- b. source_ip
- c. http_method
- d. URI
- e. status_code
- f. response_size
- g. referrer
- h. user_agent

Chuẩn hóa output về common schema.

Preprocess request nhưng vẫn giữ bản gốc.

Rule-based detector phát hiện được:

- i. SQL Injection
- j. XSS
- k. Path Traversal
- l. Command Injection
- m. Scanner user-agent

Xuất được:

- n. normalized JSONL
- o. normalized CSV
- p. features CSV
- q. alerts JSONL
- r. alerts CSV
- s. report Markdown

Có unit test cơ bản.

Không có dependency vào AI/ML training.

Dữ liệu output đủ sạch để module ML sau này dùng train TF-IDF/n-gram.

10. Phần dành riêng cho giai đoạn sau: AI/ML Training

Giai đoạn này chỉ chuẩn bị dữ liệu cho module AI/ML.

Chưa triển khai các file sau:

```
train.py
evaluate.py
ml_detector.py
vector_search.py
deep_model.py
```

Sau khi pipeline nền hoàn thành, module ML sẽ dùng:

```
outputs/normalized_logs.csv
outputs/features.csv
```

outputs/preprocessed_requests.jsonl

để train các hướng:

TF-IDF char n-gram + Logistic Regression

TF-IDF char n-gram + Linear SVM

CountVectorizer + Naive Bayes

Isolation Forest anomaly scoring

Vector search / nearest neighbor

Yêu cầu thiết kế hiện tại:

- Output CSV phải có field request text rõ ràng.
- Output phải có để join giữa normalized logs, features, alerts.
- Không trộn logic train ML vào pipeline parser.
- Rule-based output có thể dùng làm weak label nếu dataset chưa có label thật.